

Операционные системы, практика

Введение

05.09.2026

Формальные вопросы

- ▶ Занятия по субботам раз в две недели в 3389
 - ▶ Две пары практики сразу, чтобы успеть сделать что-то содержательное
 - ▶ Будем работать прежде всего на парах, домашних не будет (если только доделать что-то)
 - ▶ Работа в основном командная
- ▶ Берите с собой ноуты с qemu
- ▶ Курс на HwProj: <https://hwproj.ru/courses/50095>
- ▶ Контакты преподавателей:
 - ▶ Никита Юрьевич Ловягин: n.lovayagin@spbu.ru
 - ▶ Михаил Александрович Серов: m.serov@spbu.ru
 - ▶ Георгий Константинович Сичкар: georgy.sichkar@gmail.com
 - ▶ Антон Владимирович Черноусов: anton.chernousov@spbu.ru
 - ▶ Юрий Викторович Литвинов: y.litvinov@spbu.ru
 - ▶ Яков Александрович Кириленко: y.kirilenko@spbu.ru
 - ▶ Сергей Владимирович Погожев: s.pogozhev@spbu.ru

Критерии оценивания

- ▶ Всего примерно 7 заданий, каждое оценивается из 10 баллов
 - ▶ Можно один раз пересдать, чтобы улучшить балл
 - ▶ -3 невозможных за пропуск дедлайна
 - ▶ Делаем в основном на паре, но если пропустили, можно досдать
- ▶ Баллы за домашние работы переводятся в шкалу 0-40
- ▶ Будут две «контрольные» — индивидуальные опросы и простые задачи на листочке
 - ▶ так что даже в командах надо понимать *всё*, что делается
- ▶ Могут быть летучки с теоретическими вопросами на 5 минут, на любой паре
- ▶ В конце — теоретический зачёт по шкале 0-60

Что будет в курсе

- ▶ Базовая работа в xv6 — установка/запуск, написание простых приложений с системными вызовами, добавим свой системный вызов
- ▶ Работа с таблицей процессов
- ▶ Многопоточность и синхронизация, мьютексы
- ▶ Трассировка событий ядра, dmesg
- ▶ Межпроцессное взаимодействие
- ▶ Написание драйверов
- ▶ Файловые системы — чтение образа ext2
- ▶ Вполне возможно, что-то ещё

Практики могут немного обгонять лекции!

Краткий обзор xv6

- ▶ Учебная операционная система для курса MIT по ОС
 - ▶ <https://pdos.csail.mit.edu/6.1810/2026/index.html>
- ▶ POSIX-like
 - ▶ Не поддерживает много чего из POSIX, зато простая
 - ▶ «Почти Linux»
- ▶ Важные отличия от обычных ОС:
 - ▶ Один поток на процесс
 - ▶ Не линкуется со стандартной библиотекой C (glibc), есть своя сильно урезанная версия (ulib)
 - ▶ Собираем образ прямо с прикладной программой (как во встроенных системах)
 - ▶ Заточена под 64-битный RISC-V
- ▶ Репозиторий: <https://github.com/mit-pdos/xv6-riscv>
- ▶ Инструкция по сборке и установке:
<https://pdos.csail.mit.edu/6.1810/2026/tools.html>

Структура исходников

- ▶ kernel — монолитное ядро системы
 - ▶ entry.S — начальный загрузчик (на ассемблере RISC-V)
 - ▶ main.c — собственно инициализация системы
 - ▶ proc.c — процессы и планировщик
 - ▶ trap.c — обработка прерываний
 - ▶ syscall.c — диспетчер системных вызовов
 - ▶ console.c — код для работы с консолью
 - ▶ pipe.c — код работы с *пайпами*
 - ▶ ...
- ▶ user — программы пространства пользователя
 - ▶ sh.c — командная оболочка
 - ▶ cat.c, echo.c, grep.c, kill.c, wc.c — базовые утилиты
 - ▶ не стесняйтесь почитать их код, они очень простые!
 - ▶ ls.c, mkdir.c, ln.c, rm.c — утилиты для работы с файлами
 - ▶ user.h — заголовочный файл для системных вызовов, `ulib` и `printf`
 - ▶ его подключать в свои программы
- ▶ mkfs — утилита, собирающая загрузочный образ

Системные вызовы

- ▶ `int fork()` — создаёт новый процесс, копию текущего
- ▶ `int exit(int status)` — заканчивает текущий процесс
- ▶ `int wait(int *status)` — блокирует текущий процесс, пока не закончится дочерний
- ▶ `int pause(int n)` — блокирует текущий процесс на `n` тиков
- ▶ `int getpid()` — возвращает ID текущего процесса
- ▶ `int kill(int pid)` — завершить указанный процесс
- ▶ `int dup(int fd)` — создать новый файловый дескриптор для `fd`
- ▶ `int pipe(int p[])` — создать пайп, возвращает в `p` два дескриптора
- ▶ ...

Процессы

- ▶ У каждого процесса свой pid, можно узнать с помощью **pid_t getpid(void)**;
- ▶ Программно создать процесс — **pid_t fork(void)**;
 - ▶ Возвращает 0 для дочернего процесса и pid дочернего процесса для родительского, -1, если ошибка
 - ▶ Дочерний процесс является копией родительского, данные меняются независимо
- ▶ Завершение процесса — **void exit(int status)**;
 - ▶ return из main вызывает exit
- ▶ Дождаться завершения дочернего процесса — **pid_t wait(int *wstatus)**;
 - ▶ Если дочерних процессов несколько, ждём первого завершившегося
- ▶ Заместить текущий процесс бинарником с диска — **int exec(char *file, char *argv[])**
 - ▶ Обычно используется по схеме fork — настройка потоков ввода-вывода — exec

Ввод-вывод

- ▶ С каждым процессом ассоциирована таблица дескрипторов открытых файлов
 - ▶ Аналог FILE* из glibc, но в xv6 в пользовательских программах это просто число
- ▶ Дескриптор с индексом 0 — stdin, 1 — stdout, 2 — stderr
- ▶ Их можно подменять: close(0); open("input.txt", O_RDONLY);
 - ▶ open гарантированно выберет минимальный свободный индекс
 - ▶ в процессе всегда один поток, так что никто не «украдёт» ячейку под дескриптор

Пример: пайп

```
int p[2];
char *argv[2];
argv[0] = "wc";
argv[1] = 0;
pipe(p);
if (fork() == 0) {
    close(0); // Закрываем stdin
    dup(p[0]); // Ячейка 0 свободна, копия выходного конца пайпа "упадёт" туда
    close(p[0]); // Закрываем исходный дескриптор
    close(p[1]); // Закрываем входной конец пайпа
    exec("/bin/wc", argv); // Стартуем wc с подменённым stdin
} else {
    close(p[0]); // Закрываем выходной конец пайпа
    write(p[1], "hello world\n", 12); // Пишем в пайп
    close(p[1]); // Закрываем входной конец (иначе wc не закончит работу)
}
```

Как добавить свою программу

- ▶ Добавить файл .c в папку user (проще всего скопировать что-то типа cat)
- ▶ Не забыть подключить user.h, ещё могут быть полезны kernel/types.h и kernel/stat.h
- ▶ Реализовать там содержательную логику
 - ▶ система сборки не любит многофайловые утилиты, так что лучше всё в одном файле
- ▶ Добавить в Makefile:
`UPROGS=\ ...
$U/_filename`
- ▶ Собрать и запустить систему (make qemu)
- ▶ Вызвать вашу программу из шелла как обычно

Как добавить системный вызов

- ▶ Добавить в файл `kernel/syscall.h` константу с номером системного вызова, начинающуюся с префикса `SYS_`
- ▶ Добавить прототип функции ядра обработки системного вызова в файл `kernel/syscall.c`
 - ▶ Функция не должна иметь аргументов и должна иметь возвращаемое значение типа `uint64`
- ▶ В том же файле добавить указатель на эту функцию в таблицу системных вызовов (`syscalls`)
- ▶ Создать в `kernel` отдельный `.c` с префиксом `sys_`, где и реализовать содержательную логику
- ▶ Добавить прототип системного вызова в `user/user.h`
- ▶ Добавить функцию в `user/usys.pl`, чтобы сгенерировалась машинерия вызова из пространства пользователя
- ▶ Добавить в `Makefile` в `OBJS`: `$K/sys<ваш системный вызов>.o`
- ▶ Принимать параметры внутри системного вызова надо через функцию `argint`
 - ▶ У ядра свой стек, обычная передача параметров не работает
 - ▶ Например, заголовку `int add (int, int)`; в `user.h` соответствует `uint64 sys_add(void)` в ядре

Задача 1

На 3 балла

Написать программу, выполняющую следующие действия в xv6.

- ▶ Создается дочерний процесс, который приостанавливается (pause) 5-15 секунд и завершается с кодом возврата 1 (exit).
- ▶ Родительский процесс выводит свой идентификатор (getpid) и идентификатор дочернего процесса, затем
 - ▶ (a) просто ожидает завершения дочернего процесса (wait), после чего выводит идентификатор завершеного процесса и его код возврата, затем сам завершается (exit)
 - ▶ (b) сначала посылает дочернему процессу сигнал kill, после чего ожидает его завершения, выводит идентификатор завершеного процесса и код возврата, затем сам завершается
- ▶ В варианте (a) протестировать запуск приложения в виде фонового процесса оболочки и отправку сигнала дочернему процессу с помощью команды kill, не дожидаясь его завершения по тайм-ауту.

Задача 2

На 3 балла

Добавить в xv6 системный вызов, вычисляющий сумму двух целых чисел (своих аргументов).

- ▶ Для доступа к аргументам системного вызова использовать соответствующие функции (argint)
- ▶ Вернуть сумму чисел как результат системного вызова
- ▶ Добавить отладочный вывод
- ▶ Протестировать работу с помощью утилиты пользовательского пространства

Задача 3* (1)

На 4 балла

Написать программу, которая создает дочерний процесс и передает ему через заранее созданный анонимный канал (pipe) все аргументы командной строки с помощью write, завершая их символом перевода строки '\n', закрывает канал по завершении записи.

Дочерний процесс замещает стандартный ввод (0) выходным концом канала (pipefd[0]) с помощью dup:

```
close(0);
```

```
dup(pipefd[0]);
```

после чего канал может быть закрыт

```
close(pipefd[0]);
```

Задача 3* (2)

На 4 балла

Далее дочерний процесс замещается приложением `wc`

```
char *argv[] = {"/wc", 0};  
exec("/wc", argv);
```

Программа `wc` должна вывести подсчет строк (количества аргументов командной строки), слов и байтов в них. Родительский процесс ожидает завершения дочернего процесса (`wait`) и завершается сам (`exit`).

Задача 4

На 3 балла, бонусная

Реализовать в «настоящей» ОС (POSIX) программу, которая создает дочерний процесс и передает ему через заранее созданный анонимный канал (pipe) все аргументы командной строки с помощью write, завершая их символом перевода строки '\n', закрывает канал по завершении записи. Дочерний процесс читает данные из канала с помощью read и выводит их на стандартный вывод.

Обратите внимание! (1)

- ▶ Вызов pause в xv6 считает «ticks» (примерно 1/10 секунды, вычисляется по инструкциям процессора)
 - ▶ подсчета «настоящего» времени нет
- ▶ Закрывайте используемые концы канала и иные файлы после работы
 - ▶ ОС их закроет при завершении процесса, но лучше самим
- ▶ Заккрытие неиспользуемых концов канала обязательно, т.к. иначе может возникнуть взаимоблокировка
- ▶ Проверяйте на ошибки все используемые функции и системные вызовы!

Обратите внимание! (2)

- ▶ Вызов `exes` может завершиться с ошибкой, тогда выполняется следующий за ним код процесса
 - ▶ иначе выполняется запущенная программа, про исходный процесс «забывают»
- ▶ Оболочка `xv6` не поддерживает экранирование пробелов и других символов (кавычки и т.п.), поэтому `ws` будет возвращать равное количество слов и строк
 - ▶ можно вызывать `ws` из другой с помощью `exes`, и передать аргументы с пробелами или переводом строки
 - ▶ искусственное добавление `'\n'` к каждому аргументу в родительском процессе не гарантирует, что `ws` посчитает число аргументов исходного процесса, тем более в реальной системе

Обратите внимание! (3)

- ▶ Оболочка xv6 по каким-то причинам ограничивает количество аргументов приложения сильнее, чем ядро (10 против 32). Это можно пропатчить.
- ▶ Формально, успехом `write` считается любое положительное число выведенных байтов
 - ▶ `write` может вывести меньше данных, чем указано, и это не будет ошибкой — надо следить за реально выведенными байтами
- ▶ аналогичная ситуация с `read`: реальное количество прочитанных байтов может быть меньше размера буфера
 - ▶ чтение нужно проводить до достижения конца файла (0) или до ошибки (отрицательное значение)